# CORRIGÉ — TRAVAUX DIRIGÉS (TD)

RC-L1M-TD-PY-01 · Réservé Pass Élite

| MATIÈRE | CLASSE | DURÉE | BARÈME | CHAPITRE(S) |
|---|---|---|---|---|
| Programmation en langage Python | Licence 1 MIAGE | 1 h 30 | /20 — auto-évaluation | Entrées/sorties, conditions et boucles |

**Consignes :** Séance de travaux dirigés d'entraînement — exercices originaux rédigés par ResumeCI dans l'esprit des TD de Licence 1. Écris chaque programme sur papier ou dans un éditeur, teste-le mentalement, puis compare avec le corrigé.

## Exercice 1 — Conditions et intervalles (3 pts)

**Structure if/elif correcte 1,5 pt · Seuils exacts 1,5 pt**

*Écrire un programme Python qui lit une température saisie par l'utilisateur.*

Le programme affiche : « Gel » si elle est strictement négative, « Frais » entre 0 et 14 °C, « Doux » entre 15 et 24 °C, « Chaud » entre 25 et 34 °C et « Très chaud » à partir de 35 °C.

**✓ CORRIGÉ**

```
t = float(input("Température en °C : "))

if t < 0:
    print("Gel")
elif t <= 14:
    print("Frais")
elif t <= 24:
    print("Doux")
elif t <= 34:
    print("Chaud")
else:
    print("Très chaud")
```

Les `elif` en cascade garantissent qu'un seul message s'affiche : l'ordre des tests compte (du plus petit au plus grand seuil).

## Exercice 2 — Signe d'un produit sans le calculer (3 pts)

**Cas nul géré 1 pt · Règle des signes 2 pts**

*Écrire un programme qui lit deux nombres réels a et b.*

Le programme indique si leur produit est positif, négatif ou nul — **sans calculer le produit** a × b.

**✓ CORRIGÉ**

```
a = float(input("a = "))
b = float(input("b = "))

if a == 0 or b == 0:
    print("Le produit est nul")
elif (a > 0 and b > 0) or (a < 0 and b < 0):
    print("Le produit est positif")
else:
    print("Le produit est négatif")
```

Astuce : un produit est positif quand les deux facteurs ont le **même signe**, négatif quand ils ont des **signes contraires**, nul si l'un des deux est nul.

## Exercice 3 — Facturation par tranches (4 pts)

**Découpage en tranches 2 pts · Calcul juste 2 pts**

*Une entreprise de transport facture ses courses ainsi : prise en charge fixe de 500 FCFA, les 5 premiers kilomètres à 300 F/km, du 6e au 15e kilomètre à 250 F/km, et au-delà de 15 km à 200 F/km.*

Écrire un programme qui lit la distance parcourue (en km) et affiche le montant total de la course.

**✓ CORRIGÉ**

```
d = float(input("Distance en km : "))

prix = 500                      # prise en charge
if d <= 5:
    prix += d * 300
elif d <= 15:
    prix += 5 * 300 + (d - 5) * 250
else:
    prix += 5 * 300 + 10 * 250 + (d - 15) * 200

print("Montant de la course :", prix, "FCFA")
```

Exemple : pour 20 km → 500 + 1 500 + 2 500 + 1 000 = **5 500 FCFA**. Attention à bien découper la distance en tranches, et non appliquer un seul tarif à toute la distance.

## Exercice 4 — Boucle et statistiques (5 pts)

**Boucle de saisie 1,5 pt · Maximum 2 pts · Compteur de pairs 1,5 pt**

*Écrire un programme qui demande d'abord un entier N, puis lit N nombres entiers saisis un par un.*

À la fin de la saisie, le programme affiche la plus grande valeur lue et le nombre de valeurs paires saisies.

**✓ CORRIGÉ**

```
n = int(input("Combien de nombres ? "))

maximum = None
nb_pairs = 0
for i in range(n):
    x = int(input("Nombre " + str(i + 1) + " : "))
    if maximum is None or x > maximum:
        maximum = x
    if x % 2 == 0:
        nb_pairs += 1

print("Maximum :", maximum)
print("Valeurs paires :", nb_pairs)
```

On évite d'initialiser le maximum à 0 (faux si tous les nombres sont négatifs) : on le prend égal à la première valeur lue. Un nombre est pair quand `x % 2 == 0`.

## Exercice 5 — Test de divisibilité et diviseurs (5 pts)

**a) Diviseurs 2 pts · b) Somme stricte et test 3 pts**

*Écrire un programme qui lit un entier strictement positif n.*

a. Afficher tous ses diviseurs positifs.

b. Afficher ensuite « Nombre parfait » si n est égal à la somme de ses diviseurs stricts (diviseurs autres que n), sinon « Non parfait ». (Exemple : 6 = 1 + 2 + 3 est parfait.)

**✓ CORRIGÉ**

```
n = int(input("n = "))

somme = 0
print("Diviseurs de", n, ":")
for d in range(1, n + 1):
    if n % d == 0:
        print(d)
        if d < n:
            somme += d

if somme == n:
    print("Nombre parfait")
else:
    print("Non parfait")
```

**a)** $d$ est diviseur de $n$ quand `n % d == 0`. **b)** On cumule les diviseurs stricts (d<n) dans `somme`. Pour $n = 6$ : diviseurs 1, 2, 3, 6 et $1 + 2 + 3 = 6 \rightarrow$ **Nombre parfait**. Pour $n = 12$ : $1 + 2 + 3 + 4 + 6 = 16 \neq 12 \rightarrow$ Non parfait.